

ЗАДАНИЕ
на курсовой проект

по дисциплине «Программирование на Python (факультатив)»

Студент: _____

Группа: _____

Тема проекта: _____

Преподаватель: Кравчинский С. А.

1 Цель и задачи курсового проекта

1.1 Цель курсового проекта: показать, что студент собирает из отдельных приёмов работающую программу, а не только решает задачу в десять строк. Объём программы: примерно от 200 до 400 строк.

1.2 Задачи: описать предметную область классами и распределить ответственность между ними; разложить классы по файлам, отделив точку входа от правил программы; организовать хранение данных во внешнем файле; обеспечить устойчивость программы к неверному вводу; объяснить на защите каждое принятое решение.

2 Темы проектов

2.1 Основные темы, разбираемые на занятиях:

а) **Hangman (виселица)**: слово из файла, названные буквы, шесть попыток, виселица из символов; архитектура разбирается в уроке 7;

б) **Wordle**: слово из пяти букв, шесть попыток, отметки «буква на месте» и «буква есть, но не на этом месте»;

в) **Quiz (викторина)**: вопросы и варианты ответа из json, счёт, таблица рекордов; начало есть в задачах урока 6.

2.2 Расширенный список тем: крестики-нолики с компьютерным противником; тамагочи, у которого сытость, настроение и усталость меняются от действий игрока; менеджер задач с сохранением в json; банковский симулятор со счетами и переводами; библиотека книг с выдачей, возвратом и поиском по автору; дневник тренировок со статистикой.

2.3 Своя тема согласуется с преподавателем заранее: нужны одна фраза о том, что делает программа, и список классов с их ответственностью. Тема годится, если из неё выходит не меньше трёх содержательных классов.

3 Требования к программе

Требования 3.1–3.9 обязательны. Невыполнение любого из них возвращает проект на доработку.

3.1 Программа консольная, доведена до рабочего состояния и проходит сценарий целиком: запуск, полная партия, завершение.

3.2 Классов предметной области не меньше трёх, каждый в отдельном файле в папке `models/`. Точка входа `main.py` содержит `if __name__ == '__main__':`.

3.3 В каждом классе определён метод `__str__`, возвращающий осмысленную строку.

3.4 Инкапсуляция: поля, которые нельзя менять снаружи, защищены (`_attempts`) или приватны (`__balance`); есть хотя бы одно `@property` с проверкой значения в `setter`.

3.5 Программа не завершается аварийно при неверном вводе: нечисловая строка, пустая строка и число вне диапазона обрабатываются.

3.6 Данные хранятся в файле `txt` или `json`: список слов, вопросы, рекорды, настройки. Файл читается с параметром `encoding='utf-8'`.

3.7 Глобальные переменные не используются; константы ЗАГЛАВНЫМИ допустимы (`MAX_ATTEMPTS = 6`).

3.8 Выполнено одно требование на выбор: а) наследование с полиморфизмом; б) хотя бы одна внешняя библиотека.

3.9 Имена по PEP 8: классы `CapWords`, функции и переменные `snake_case`.

3.10 Выполнение обязательных требований проверяется на защите способом, указанным в таблице 1.

Таблица 1 — Проверка обязательных требований на защите

Требование	Как проверяется на защите
3.1 Рабочая программа	Запускаем <code>python main.py</code> , играем одну полную партию
3.2 Классы и структура файлов	Смотрим дерево проекта, открываем <code>main.py</code>

Требование	Как проверяется на защите
3.3 Метод <code>__str__</code>	Печатаем объект каждого класса: ждём осмысленную строку, а не <code><models.game.Game object at 0x...></code>
3.4 Инкапсуляция	Присваиваем через <code>property</code> неверное значение, ждём понятную ошибку
3.5 Устойчивость к вводу	Вводим <code>abc</code> , пустую строку, <code>-5, 999</code> ; <code>traceback</code> на экране равен незачёту
3.6 Хранение данных в файле	Меняем строку в файле данных, перезапускаем, видим изменение
3.7 Нет глобальных переменных	Читаем верхний уровень модулей: только импорты, константы, классы и функции
3.8 Требование на выбор	Вариант а): базовый класс, потомки, общий вызов. Вариант б): импорт и <code>requirements.txt</code>
3.9 Имена по PEP 8	Читаем код: <code>PyCharm</code> подчёркивает нарушения сам

4 Что повышает оценку

4.1 Сверх обязательных требований оценку повышает:

а) выполнение обоих вариантов требования 3.8: и наследование с полиморфизмом, и внешняя библиотека;

б) магические методы сверх `__str__`: `__eq__`, `__lt__`, `__len__`, `__add__` (таблица из урока 7);

в) аннотации типов и `docstring`: `def guess(self, letter: str) -> bool`: и строка о том, что метод делает;

г) проверки логики без участия человека: отдельный файл с `assert` для ключевых методов;

д) аккуратный вывод: выравнивание, рамки, цвет через библиотеку `rich` из урока 5;

е) сохранение состояния между запусками: рекорды, статистика, незаконченная партия;

ж) своя тема вместо готовой из списка.

5 Состав и оформление сдаваемых материалов

5.1 Сдаётся папка проекта или архив с ней: `main.py`, папка `models/` с классами и файлы данных, без которых программа не запустится.

5.2 Файл `README.md` содержит: что делает программа, дословную команду запуска, перечень классов с их ответственностью.

5.3 Файл `requirements.txt` включается при использовании внешних библиотек; без них файл не нужен.

5.4 Структура папок соответствует приложению А: точка входа в корне проекта, классы в папке `models/`, данные отдельными файлами.

5.5 Служебные папки `.venv` и `__pycache__` в архив не включаются: это не ваш код.

5.6 Оформление кода по PEP 8: отступ четыре пробела, классы `CapWords`, функции и переменные `snake_case`, пустая строка между методами.

6 Порядок защиты и критерии оценки

6.1 Защита проводится в следующем порядке:

а) студент запускает программу и играет одну полную партию до конца;

б) показывает дерево проекта и говорит по одной фразе про каждый класс;

в) открывает два-три места в коде по просьбе преподавателя и объясняет их;

г) отвечает на вопросы по коду и по устройству программы;

д) вносит небольшое изменение прямо на защите: другое число попыток, дополнительная проверка ввода, другой текст вывода. PyCharm, документация и интернет доступны.

6.2 Примерные вопросы:

– что такое `self` в этом методе и что будет, если убрать его из параметров;

– почему это поле защищённое и что сломается при изменении снаружи;

– где здесь `@property` и что именно проверяет `setter`;

- что вернёт `__str__` этого объекта и в какой момент он вызывается;
- что произойдёт, если файла с данными нет, и где это обрабатывается;
- почему `Game` содержит `Player`, а не наследуется от него;
- что делает строка, выбранная преподавателем;
- как добавить ещё один режим и какие файлы придётся изменить.

6.3 Проект зачтён, если выполнены требования 3.1–3.9, студент объясняет любой фрагмент своего кода и внесённое на защите изменение работает.

6.4 Проект возвращается на доработку, если не выполнено любое из требований 3.1–3.9, программа выводит `traceback` на проверочном вводе или студент не может объяснить свой код: последний случай разбирается по разделу 7.

6.5 Оценка повышается за пункты раздела 4, если сделанное работает и студент может это объяснить.

7 Использование ИИ и заимствования

7.1 Пользоваться ИИ-помощником, поиском и чужими примерами разрешается. Условие ровно одно: **каждая строка проекта понятна его автору.**

7.2 Любая строка объясняется вслух: что делает, зачем нужна, что сломается без неё.

7.3 Код, который студент не смог объяснить на защите, считается чужим. Это незачёт.

7.4 Скопированный целиком чужой проект (из интернета, у однокурсника, полученный одним запросом у помощника) равен незачёту независимо от того, работает ли он.

7.5 Непонятную строку, предложенную помощником, следует удалить и написать проще: простой понятный код лучше сложного и чужого. Помощника разумно просить не готовый код, а объяснение ошибки, разбор своего варианта или подсказку.

7.6 Код с занятий, стандартная библиотека и внешние библиотеки заимствованием не считаются: это инструменты, а не чужая работа.

Приложение А. Пример структуры проекта

Устройство проекта показано на примере темы Hangman. Имена файлов и классов могут быть другими, состав папок обязателен.

hangman/	
main.py	создаёт объекты, запускает игру
models/	
__init__.py	делает папку пакетом
word_manager.py	класс WordManager
player.py	класс Player
game.py	класс Game
display.py	класс Display
words.txt	слова, по одному в строке
README.md	описание и команда запуска
requirements.txt	только при внешних библиотеках

Назначение классов:

- **WordManager** знает только про файл со словами: читает `words.txt` (урок 5), отдаёт случайное слово через `random.choice`, отбирает слова нужной длины;
- **Player** помнит имя игрока, оставшиеся попытки и названные буквы; `attempts_left` доступен только для чтения (property без setter, урок 6), уменьшает его метод `lose_attempt()`;
- **Game** держит правила: загаданное слово, объекты `Player` и `WordManager` внутри себя (композиция, урок 7), `guess(letter)` возвращает, была ли буква в слове, `is_won` и `is_lost` заканчивают партию; ничего не печатает;
- **Display** единственный говорит с человеком: печатает маску слова, оставшиеся попытки и виселицу, спрашивает букву и проверяет ввод (одна буква, не цифра, ещё не называлась), на плохой ввод спрашивает снова, как `ask_int()` из урока 5;
- **main.py** собирает объекты и запускает `game.run()`: правил игры в нём нет, пять-семь строк под `if __name__ == '__main__':`.

Отдельный класс `Display` нужен для защиты: когда попросят сменить приветствие, правится один файл, а правила игры остаются нетронутыми. Рабочий образец обязательной структуры лежит рядом с этим документом: `course_project/example_project`, игра «Угадай число».

Приложение Б. Чеклист самопроверки перед сдачей

Чеклист проходят на чужом компьютере: там не окажется случайно нужного файла.

- ☐ Программа запускается командой `python main.py` из папки проекта, а не только кнопкой в PyCharm.
- ☐ Одна полная партия проходится до конца: и выигрыш, и проигрыш.
- ☐ Классов предметной области не меньше трёх, каждый в своём файле в `models/`; в `main.py` есть `if __name__ == '__main__':` и нет правил игры.
- ☐ В каждом классе определён `__str__`, печать объекта даёт читаемую строку.
- ☐ Есть хотя бы одно `@property` с проверкой в `setter`, и неверное значение оно не пропускает.
- ☐ Ввод `abc`, пустая строка и число вне диапазона не роняют программу.
- ☐ Файл данных читается с `encoding='utf-8'`, его отсутствие даёт понятное сообщение, а не `traceback`.
- ☐ Глобальных переменных нет, константы записаны ЗАГЛАВНЫМИ.
- ☐ Выполнен пункт на выбор: наследование с полиморфизмом либо внешняя библиотека в `requirements.txt`.
- ☐ В `README.md` написано, что делает программа и как её запустить.
- ☐ Любую строку проекта вы можете объяснить вслух.